

МИНОБРНАУКИ РОССИИ
ФГБОУ ВО «ИжГТУ имени М.Т. Калашникова»
Факультет «Информационные технологии»
Кафедра «Программное обеспечение»

КУРСОВАЯ РАБОТА

по дисциплине:

«Базы данных»

на тему: «Разработка базы данных для автоматизации аэропорта»

Выполнил(а):

студент гр. Б24-191-3

В.Д. Мошкин

Проверил:

к.т.н, доцент

А.В. Старыгин

Ижевск, 2026

ОГЛАВЛЕНИЕ

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ	3
ССЫЛКА НА КОД В ПУБЛИЧНОМ РЕПОЗИТОРИИ.....	4
ВВЕДЕНИЕ	5
ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	7
1.1. База данных PostgreSQL.....	7
1.2. Структура базы данных.....	7
1.3. Обоснование выбора языка программирования для разработки клиентского приложения.	8
1.3. Общие сведения о разработанном приложении.....	9
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	11
2.1. Структура проекта	11
2.2. Реализации базы данных	13
2.3. Схема базы данных	14
2.4. Реализация клиентского приложения	17
ЗАКЛЮЧЕНИЕ.....	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	22

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Тема курсовой работы: «Разработка базы данных для автоматизации аэропорта»

Цель курсовой работы: Разработать клиентское приложение для автоматизации аэропорта. Приложение позволяет сотрудникам аэропорта вести учет аэропортов, вылетов, персонала, самолетов, маршрутов, а также создавать и формировать отчеты по списку ближайших рейсов, по конечному пункту назначения выдавать список возможных маршрутов.

Задачи курсовой работы:

- Спроектировать и создать базу данных в PostgreSQL
- Разработать клиентское приложение для работы с созданной базой данных

ССЫЛКА НА КОД В ПУБЛИЧНОМ РЕПОЗИТОРИИ

Публичный репозиторий: GitHub

Ссылка: <https://github.com/Paradise4412/DatabasesIstuMoshkinVD#>

ВВЕДЕНИЕ

В современных условиях эффективная организация работы аэропорта невозможна без использования современных информационных систем и технологий обработки данных. Аэропорт представляет собой сложный инфраструктурный объект, в котором ежедневно выполняется большое количество взаимосвязанных процессов: ведётся учёт рейсов, регистрация пассажиров, оформление билетов, контроль багажа, распределение персонала, координация работы служб и обслуживание воздушных судов. Все эти процессы требуют высокой точности, оперативности и согласованности действий, что делает применение хорошо организованной базы данных необходимым условием стабильного функционирования аэропорта.

Актуальность темы курсовой работы определяется тем, что в условиях постоянного увеличения объёма пассажиропотока, роста числа рейсов и повышения требований к качеству обслуживания возрастает необходимость в автоматизации основных процессов аэропорта. Использование разрозненных источников информации, бумажного документооборота или недостаточно структурированных электронных данных значительно снижает эффективность работы, затрудняет быстрый поиск необходимых сведений и повышает вероятность возникновения ошибок. Это может отрицательно влиять как на внутреннюю организацию деятельности аэропорта, так и на уровень обслуживания пассажиров.

Особую значимость разработка базы данных приобретает потому, что деятельность аэропорта связана с обработкой большого количества разнообразной информации, которая должна быть логически упорядочена, защищена и доступна для оперативного использования. К такой информации относятся сведения о пассажирах, рейсах, билетах, маршрутах, самолётах, экипаже, сотрудниках различных служб и багаже. При отсутствии единой базы данных управление этими сведениями становится менее эффективным, а выполнение повседневных операций требует больших временных затрат. Именно поэтому проектирование базы данных выступает важным этапом создания автоматизированной системы управления деятельностью аэропорта.

В рамках курсовой работы рассматриваются вопросы, связанные с анализом предметной области и проектированием базы данных, предназначенной для автоматизации работы аэропорта. Особое внимание уделяется выделению основных сущностей предметной области, определению связей между ними, формированию структуры таблиц, выбору ключевых атрибутов и обеспечению целостности данных. Кроме того, рассматривается практическое значение базы данных как инструмента, позволяющего упорядочить хранение

информации, ускорить её обработку и повысить эффективность выполнения основных операций.

Практическая значимость данной работы заключается в том, что разработанная база данных может быть использована для повышения эффективности управления информацией, сокращения времени на выполнение типовых операций, уменьшения вероятности ошибок при обработке данных и улучшения качества обслуживания пассажиров. Также подобная база данных может стать основой для дальнейшего расширения функциональных возможностей системы и создания полноценной автоматизированной информационной среды аэропорта.

Разработка базы данных для автоматизации аэропорта является важной и востребованной задачей, она позволяет обеспечить более высокий уровень организации работы, надёжность хранения данных, удобство их использования и оптимизацию процессов.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. База данных PostgreSQL

PostgreSQL представляет собой объектно-реляционную систему управления базами данных, предназначенную для надёжного хранения, обработки и анализа структурированной информации. Данная СУБД поддерживает язык SQL, механизм транзакций, контроль целостности данных, индексацию, представления, триггеры и серверные функции. Благодаря этим возможностям PostgreSQL широко применяется при разработке информационных систем различного назначения, в том числе систем, связанных с хранением и обработкой больших объёмов взаимосвязанных данных.

Одним из важных преимуществ PostgreSQL является поддержка развитого механизма ограничения целостности. В структуре базы данных могут использоваться первичные и внешние ключи, проверки значений с помощью CHECK, значения по умолчанию DEFAULT, а также каскадные правила обновления и удаления записей. Кроме того, PostgreSQL позволяет создавать индексы для ускорения поиска данных, представления для упрощения доступа к сведениям и функции на языке PL/pgSQL для реализации прикладной логики непосредственно на уровне базы данных.

В курсовой работе PostgreSQL используется для реализации базы данных автоматизированной системы учёта аэропортов, воздушных судов, персонала и маршрутов. В разработанной структуре базы данных определены основные таблицы airports, positions, crews, services, airplanes, personnel, routes и transit_routes.

1.2. Структура базы данных

В разработанной структуре базы данных определены основные таблицы airports, positions, crews, services, airplanes, personnel, routes и transit_routes.

Таблица airports предназначена для хранения сведений об аэропортах и содержит такие атрибуты, как airport_code, name, inn, address, city, country, phone, created_at, departures_count и arrivals_count. Для обеспечения корректности данных в данной таблице реализованы ограничения airports_inn_chk, airports_dep_chk и airports_arr_chk.

Таблицы positions, crews и services используются как справочники и содержат сведения о должностях, экипажах и службах соответственно. Таблица airplanes предназначена для хранения информации о воздушных судах и включает поля airport_code, airplane_code, name, model, seats, in_service и routes_assigned_count. Для неё реализовано внешнее ограничение airplanes_airport_fk, обеспечивающее связь с таблицей airports, а также ограничение airplanes_routes_cnt_chk.

Таблица personnel служит для хранения данных о сотрудниках аэропорта. В ней используются поля airport_code, person_inn, full_name, position_code, crew_code, service_code и hired_at. Для данной таблицы реализованы ограничения personnel_inn_chk,

personnel_airport_fk, personnel_position_fk, personnel_crew_fk и personnel_service_fk, что обеспечивает логическую согласованность хранимых сведений.

Таблица routes предназначена для описания маршрутов полётов. В ней хранятся route_code, start_airport_code, end_airport_code, flight_hours, airplane_airport_code, airplane_code, departure_time, flight_no, notes и transit_stop_count. Для контроля корректности данных используются ограничения routes_hours_chk, routes_transit_cnt_chk, routes_start_fk, routes_end_fk и routes_airplane_fk. Таблица transit_routes реализует учёт промежуточных остановок по маршрутам и включает поля route_code, stop_no, stand_no, stop_airport_code, arrival_time и departure_time. Для неё предусмотрены ограничения transit_routes_route_fk, transit_routes_airport_fk и transit_routes_times_chk.

Для повышения производительности выполнения запросов в базе данных созданы индексы idx_airports_city, idx_airplanes_airport, idx_personnel_airport, idx_personnel_service, idx_routes_departure, idx_routes_start_end и idx_transit_route. Их использование позволяет ускорить поиск и выборку данных по наиболее часто используемым полям.

Существенной частью реализации является использование серверных функций и триггеров. В работе созданы функции refresh_airport_route_totals, refresh_airplane_route_usage и refresh_route_transit_stop_count, предназначенные для автоматического обновления производных количественных показателей. Для запуска этих процедур при изменении данных реализованы триггерные функции trg_transit_routes_refresh_route_count и trg_routes_refresh_airports_and_plane, а также триггеры tr_transit_routes_denorm и tr_routes_denorm. Благодаря этому значения departures_count, arrivals_count, routes_assigned_count и transit_stop_count автоматически поддерживаются в актуальном состоянии.

Для удобства получения итоговой информации в базе данных созданы представления v_airports_catalog, v_route_schedule и v_airports_departures_agg. Представление v_airports_catalog содержит каталог аэропортов, v_route_schedule формирует расписание маршрутов с названиями аэропортов и самолётов, а v_airports_departures_agg предоставляет агрегированные сведения о количестве маршрутов вылета и числе транзитных остановок.

1.3. Обоснование выбора языка программирования для разработки клиентского приложения.

В качестве основного средства разработки клиентского приложения был выбран язык программирования Python. Python представляет собой высокоуровневый интерпретируемый язык общего назначения, ориентированный на повышение скорости разработки, читаемости кода и удобства сопровождения программных продуктов. Он был создан с целью предоставить разработчику простой и понятный синтаксис, который позволяет сосредоточиться не на технических деталях реализации, а на логике решаемой задачи.

Одной из ключевых особенностей Python является его лаконичный и наглядный синтаксис. В отличие от многих других языков программирования, Python позволяет записывать программные конструкции в компактной и легко воспринимаемой форме. Благодаря этому он широко применяется как в учебных целях, так и в профессиональной разработке программного обеспечения различного уровня сложности.

Важным преимуществом Python является наличие большого количества библиотек и готовых модулей, которые расширяют стандартные возможности языка. За счет этого разработчик может использовать уже существующие средства для работы с базами данных, обработки файлов, сетевого взаимодействия, построения интерфейсов и выполнения аналитических вычислений. Наличие развитой экосистемы значительно ускоряет процесс разработки и уменьшает объем кода, который необходимо создавать вручную.

Для разработки клиентского приложения Python оказался особенно удобен по нескольким причинам. Во-первых, он предоставляет простые и надежные средства для организации взаимодействия с реляционными базами данных. Во-вторых, модульный подход, поддерживаемый языком, позволяет разделить программу на отдельные логические части, например интерфейс пользователя, функции доступа к данным и служебные процедуры. В-третьих, Python хорошо подходит для создания консольных приложений, где требуется реализовать меню, ввод данных, проверку корректности значений и вывод результатов запросов.

Выбор Python в качестве языка разработки клиентского приложения обусловлен его простотой, универсальностью, широкими возможностями, развитой экосистемой библиотек и удобством работы с базами данных. Использование данного языка позволяет сократить время разработки, обеспечить хорошую читаемость программного кода и создать приложение, которое легко модифицировать, расширять и сопровождать в дальнейшем.

1.3. Общие сведения о разработанном приложении

Клиентское приложение разработано на языке Python и представляет собой консольную программу для взаимодействия с базой данных PostgreSQL. Такой подход позволяет реализовать прикладной интерфейс без использования графической оболочки, сосредоточив внимание на логике работы с данными, выполнении CRUD-операций, формировании отчетов и обеспечении корректного взаимодействия с серверной частью базы данных.

Клиентское приложение построено по модульному принципу. Основная логика взаимодействия с базой данных сосредоточена в отдельном классе, который отвечает за подключение, получение структуры таблиц, выборку данных, вставку, обновление и удаление записей. Пользовательский интерфейс реализован отдельно и отвечает за отображение меню, прием команд пользователя и вызов соответствующих методов работы с данными.

Приложение поддерживает универсальную работу с таблицами базы данных. Для этого используются запросы к системным представлениям `information_schema` и служебным каталогам PostgreSQL, позволяющие получить перечень таблиц, сведения о столбцах, типах данных, признаке обязательности, значениях по умолчанию и составе первичных ключей. Такой подход делает приложение менее зависимым от жестко зафиксированной структуры базы данных и позволяет применять единый механизм обработки для разных таблиц.

Разработанное приложение обеспечивает выполнение основных операций обработки данных. Пользователь может просматривать содержимое таблиц, добавлять новые записи, изменять существующие и удалять их. Для таблицы маршрутов реализован специальный режим ввода, при котором одновременно добавляется основная запись о маршруте и набор

связанных транзитных стоянок. Такая операция выполняется в рамках одной транзакции, что гарантирует согласованность данных.

Помимо операций редактирования, приложение предоставляет возможность формирования аналитических отчетов. В частности, реализованы запросы для получения статистики по количеству мест размещения в разных населенных пунктах, средней стоимости проживания по типам мест размещения и перечня маршрутов с указанием числа транзитных стоянок. Наличие таких функций делает клиентское приложение не только средством ввода данных, но и инструментом анализа информации, хранящейся в базе данных.

Интерфейс приложения имеет консольный формат и организован в виде системы меню. Пользователю предоставляются понятные варианты выбора действий, сообщения о результатах операций и уведомления об ошибках. При вводе данных выполняется базовая проверка значений, а при завершении работы предусмотрена корректная обработка прерывания ввода. Это делает использование приложения удобным и понятным даже при отсутствии графической оболочки.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Структура проекта

Проект состоит из нескольких файлов, каждый из которых отвечает за отдельную часть системы: описание базы данных, начальное заполнение, доступ к PostgreSQL и реализацию клиентского консольного приложения. Такая структура делает проект понятным, удобным для запуска и дальнейшего сопровождения.

В корне проекта расположены основные файлы: `schema.sql`, `seed.sql`, `db.py`, `app.py`, `README.md` и вспомогательный файл `postgres.session.sql`.

Файл `README.md` содержит общее описание проекта, инструкцию по установке зависимостей, созданию базы данных, загрузке структуры и тестовых данных, а также сведения о составе проекта и основных возможностях программы. Этот файл нужен для первого знакомства с системой и быстрого запуска приложения.

Файл `schema.sql` предназначен для создания и обновления структуры базы данных. В нём описаны все основные объекты PostgreSQL: таблицы, первичные и внешние ключи, ограничения целостности, индексы, функции, триггеры и представления. Именно этот файл формирует логическую и физическую структуру базы данных аэропортов и маршрутов. Внутри него создаются таблицы `airports`, `airplanes`, `positions`, `crews`, `services`, `personnel`, `routes` и `transit_routes`. Также в нём создаются индексы для ускорения выборок, представления для упрощения чтения данных и триггеры для автоматического обновления денормализованных счётчиков.

Файл `seed.sql` используется для начального заполнения базы данных тестовыми данными. Он очищает таблицы, перезапускает значения автоинкрементных полей и добавляет примеры аэропортов, самолётов, должностей, экипажей, служб, сотрудников, маршрутов и транзитных остановок. Благодаря этому после запуска проекта база уже содержит демонстрационные данные, на которых можно сразу проверять работу клиентского приложения и формируемых отчётов.

Файл `db.py` реализует слой доступа к данным. В нём сосредоточена логика подключения к базе данных и выполнения SQL-запросов. Основным элементом этого файла является класс `Database`, который инкапсулирует работу с PostgreSQL. Через него выполняются запросы на чтение, вставку, обновление и удаление данных. Также в этом модуле находятся методы для получения списка таблиц, описания столбцов, первичных ключей и другая вспомогательная логика. Отдельно реализована функция `_split_sql_statements()`, которая разбивает SQL-скрипт на отдельные команды, корректно учитывая кавычки. Таким образом, `db.py` выступает промежуточным слоем между приложением и базой данных.

Файл `app.py` содержит основную реализацию клиентского приложения. Это главный исполняемый модуль проекта, который запускается пользователем из консоли. В данном файле реализовано главное меню программы, переход между режимами работы, универсальный режим просмотра таблиц, CRUD-операции, форма ввода данных сразу в две связанные таблицы и система отчётов.

Структурно файл `app.py` можно разделить на несколько логических частей:

Первая часть отвечает за служебные функции интерфейса: чтение команд, ввод значений, преобразование типов, вывод строк таблиц и обработку пользовательских действий. Эти функции обеспечивают взаимодействие пользователя с программой через консоль.

Вторая часть реализует режим работы с таблицами. Пользователь может выбрать одну из таблиц базы данных и перейти в универсальный режим просмотра записей. В этом режиме доступны навигация по строкам, переход между страницами, сортировка, фильтрация, поиск, обновление выборки, добавление новой записи, редактирование существующей и удаление строки. Такой подход позволяет использовать один и тот же механизм интерфейса для разных таблиц без необходимости писать отдельный экран для каждой сущности.

Третья часть связана с обработкой составной формы 1:M, то есть формы для добавления маршрута и связанных с ним транзитных стоянок. Эту задачу выполняет функция `_create_route_with_transit()`. Сначала пользователь вводит данные маршрута в таблицу `routes`, после чего в рамках одной транзакции могут быть добавлены связанные записи в таблицу `transit_routes`. Такой механизм демонстрирует работу с отношением «один ко многим» и обеспечивает целостность данных: если при вводе возникает ошибка, операция может быть отменена целиком.

Четвёртая часть отвечает за формирование отчётов. В приложении реализовано отдельное меню отчётов, где пользователь может выбрать нужный аналитический запрос. Среди них есть отчёт по возможным маршрутам к заданному конечному пункту, отчёт по ближайшим рейсам и отчёт по персоналу с группировкой по службам и должностям. Для каждого отчёта предусмотрен ввод параметров, например фильтра по аэропорту, окна времени или режима сортировки. Это делает отчёты не статическими, а интерактивными.

Такое разделение проекта на уровни является удачным, потому что каждая часть отвечает только за свою задачу. SQL-скрипты описывают и заполняют базу данных, Python-модуль `db.py` обеспечивает техническую работу с PostgreSQL, а `app.py` реализует пользовательское поведение и бизнес-логику приложения. Благодаря этому проект легко понимать, тестировать и расширять.

Структура проекта построена по принципу разделения ответственности. Это позволяет отдельно развивать базу данных, клиентское приложение и документацию, не смешивая все элементы в одном файле. Такое решение повышает читаемость кода, упрощает сопровождение системы и делает проект более профессионально организованным.

2.2. Реализации базы данных

База данных включает несколько взаимосвязанных таблиц. Таблица аэропортов содержит коды, наименования, город, страну, адрес, а также расчетные показатели количества вылетов и прилетов. Таблица самолетов хранит сведения о модели, производителе, вместимости, дальности полета и статусе эксплуатации. Таблица сотрудников содержит фамилию, имя, должность, дату приема на работу и контактные данные. Таблица маршрутов связывает аэропорты отправления и прибытия, назначенный самолет, пилота и дату выполнения рейса. Таблица транзитных остановок предназначена для хранения промежуточных пунктов маршрута и их последовательности.

При проектировании структуры были определены первичные и внешние ключи, обеспечивающие логическую связанность данных между таблицами. Дополнительно были заданы ограничения NOT NULL, UNIQUE и CHECK, которые предотвращают ввод некорректных значений и поддерживают целостность данных. Например, контролируется положительное значение вместимости самолета, допустимость количества транзитных остановок и невозможность совпадения аэропортов отправления и прибытия в одном маршруте.

Для повышения производительности были созданы индексы по наиболее часто используемым полям. Индексирование применялось к внешним ключам, полям поиска и уникальным кодам. Это позволяет ускорить выборку данных, соединение таблиц и выполнение фильтрации, что особенно важно при увеличении объема информации.

На основе функций были созданы триггеры, автоматически срабатывающие при добавлении, изменении или удалении записей в таблице маршрутов. Их задача заключается в том, чтобы после каждого изменения пересчитывались показатели, связанные с аэропортами отправления и прибытия. Благодаря этому данные остаются актуальными без необходимости выполнять обновление вручную.

Для упрощения работы с данными были созданы представления. Одно из них содержит справочную информацию об аэропортах вместе с основными расчетными показателями. Другое объединяет сведения о маршрутах, аэропортах отправления и прибытия, а также назначенных самолетах, что делает просмотр расписания более удобным. Также было создано агрегированное представление, показывающее количество исходящих маршрутов и

суммарное число транзитных остановок по каждому аэропорту отправления. Представления скрывают сложность SQL-запросов и делают доступ к данным более удобным.

2.3. Схема базы данных

База данных предназначена для учета аэропортов, самолетов, персонала, маршрутов и транзитных остановок. База данных содержит 8 основных таблиц:

Таблица `airports`: Содержит информацию об аэропортах. Поля: `airport_code` — код аэропорта, `VARCHAR(10)`, первичный ключ, `name` — название аэропорта, `TEXT`, обязательное поле, `inn` — ИНН, `VARCHAR(12)`, обязательное поле, `address` — адрес, `TEXT`, `city` — город, `TEXT`, `country` — страна, `TEXT`, `phone` — телефон, `TEXT`, `created_at` — дата и время создания записи, `TIMESTAMPTZ`, по умолчанию `now()`, `departures_count` — количество вылетов из аэропорта, `INTEGER`, по умолчанию 0, `arrivals_count` — количество прилетов в аэропорт, `INTEGER`, по умолчанию 0. Ограничения: `airport_code` — уникален, так как это первичный ключ, `inn` должен содержать 10 или 12 цифр, `departures_count` ≥ 0 , `arrivals_count` ≥ 0 .

Таблица `positions`: Содержит должности сотрудников. Поля: `position_code` — код должности, `VARCHAR(20)`, первичный ключ, `name` — название должности, `TEXT`, обязательное, уникальное.

Таблица `crews`: Содержит справочник экипажей. Поля: `crew_code` — код экипажа, `VARCHAR(20)`, первичный ключ, `name` — название экипажа, `TEXT`, обязательное, уникальное. Используется для привязки сотрудников к экипажам.

Таблица `services`: Содержит справочник служб. Поля: `service_code` — код службы, `VARCHAR(20)`, первичный ключ, `name` — название службы, `TEXT`, обязательное, уникальное.

Таблица `airplanes`: Содержит сведения о самолетах. Поля: `airport_code` — код аэропорта, к которому относится самолет, `VARCHAR(10)`, обязательное поле, `airplane_code` — код самолета, `VARCHAR(20)`, обязательное поле, `name` — название самолета, `TEXT`, обязательное поле, `model` — модель самолета, `TEXT`, `seats` — количество мест, `INTEGER`, `in_service` — находится ли самолет в эксплуатации, `BOOLEAN`, по умолчанию `TRUE`, `routes_assigned_count` — количество маршрутов, назначенных самолету, `INTEGER`, по умолчанию 0. Составной первичный ключ: (`airport_code`, `airplane_code`). `airport_code` $\rightarrow$ `airports(airport_code)`, `seats` > 0 , если значение указано, `routes_assigned_count` ≥ 0 .

Таблица `personnel`: Содержит сведения о персонале аэропорта. Поля: `airport_code` — код аэропорта, к которому относится сотрудник, `VARCHAR(10)`, обязательное поле, `personnel_id` — идентификатор сотрудника, `SERIAL`, обязательное поле, `full_name` — ФИО сотрудника, `TEXT`, обязательное поле, `birth_date` — дата рождения, `DATE`, обязательное поле, `hire_date` —

дата приема на работу, DATE, обязательное поле, salary — заработная плата, NUMERIC(12,2), position_id — идентификатор должности, INTEGER, обязательное поле, crew_id — идентификатор экипажа, INTEGER, service_id — идентификатор службы, INTEGER. Составной первичный ключ: (airport_code, personnel_id). Внешние ключи: airport_code → airports(airport_code), position_id → positions(position_id), crew_id → crews(crew_id), service_id → services(service_id). Проверки: salary ≥ 0, hire_date ≥ birth_date.

Таблица routes: Содержит сведения о маршрутах. Поля: route_code — код маршрута, VARCHAR(20), обязательное поле, flight_no — номер рейса, VARCHAR(20), обязательное поле, departure_time — время отправления, TIMESTAMP, обязательное поле, flight_hours — продолжительность полета в часах, NUMERIC(5,2), start_airport_code — код аэропорта отправления, VARCHAR(10), обязательное поле, end_airport_code — код аэропорта прибытия, VARCHAR(10), обязательное поле, airplane_airport_code — код аэропорта, к которому относится самолет, VARCHAR(10), обязательное поле, airplane_code — код самолета, VARCHAR(20), обязательное поле, transit_stop_count — количество транзитных остановок, INTEGER, по умолчанию 0. Первичный ключ: route_code. Внешние ключи: start_airport_code → airports(airport_code), end_airport_code → airports(airport_code), (airplane_airport_code, airplane_code) → airplanes(airport_code, airplane_code). Проверки: flight_hours > 0, если значение указано, transit_stop_count ≥ 0, start_airport_code ≠ end_airport_code.

Таблица transit_routes: Содержит сведения о транзитных остановках маршрутов. Поля: route_code — код маршрута, VARCHAR(20), обязательное поле, stop_order — порядковый номер остановки, INTEGER, обязательное поле, airport_code — код транзитного аэропорта, VARCHAR(10), обязательное поле, stop_duration_hours — длительность остановки в часах, NUMERIC(5,2). Составной первичный ключ: (route_code, stop_order). Внешние ключи: route_code → routes(route_code), airport_code → airports(airport_code). Проверки: stop_order > 0, stop_duration_hours ≥ 0, если значение указано.

Связи между таблицами:

- Один аэропорт (airports) может иметь много самолетов (airplanes)
- Один аэропорт (airports) может иметь много сотрудников (personnel)
- Один аэропорт (airports) может быть начальным или конечным пунктом во многих маршрутах (routes)
- Один самолет (airplanes) может использоваться во многих маршрутах (routes)
- Один маршрут (routes) может иметь много транзитных остановок (transit_routes)
- Одна должность (positions) может быть назначена многим сотрудникам (personnel)

- Один экипаж (crews) может включать многих сотрудников (personnel)
- Одна служба (services) может включать многих сотрудников (personnel)

Индексы:

Для ускорения поиска созданы следующие индексы:

- idx_airports_city по полю airports(city)
- idx_airplanes_airport по полю airplanes(airport_code)
- idx_personnel_airport по полю personnel(airport_code)
- idx_personnel_service по полю personnel(service_code)
- idx_routes_departure по полю routes(departure_time)
- idx_routes_start_end по полям routes(start_airport_code, end_airport_code)
- idx_transit_route по полю transit_routes(route_code)

Функции:

Функция refresh_airport_route_counts(p_airport_code): Пересчитывает количество маршрутов вылета и прибытия для указанного аэропорта.

Функция refresh_airplane_route_usage(p_airport_code, p_airplane_code): Пересчитывает количество маршрутов, назначенных указанному самолету.

Функция refresh_route_transit_stop_count(p_route_code): Пересчитывает количество транзитных остановок для указанного маршрута.

Триггеры:

Триггер tr_transit_routes_denorm: Срабатывает после INSERT, UPDATE, DELETE в таблице transit_routes. Назначение: автоматически обновляет количество транзитных остановок у маршрута.

Триггер tr_routes_denorm: Срабатывает после INSERT, UPDATE, DELETE в таблице routes. Назначение: автоматически обновляет число вылетов и прилетов аэропортов, а также количество маршрутов, назначенных самолету.

Представления:

Представление v_airports_catalog: Содержит сведения об аэропортах для удобного просмотра. Поля: airport_code, name, inn, address, city, country, phone, created_at, departures_count, arrivals_count.

Представление v_route_schedule: Содержит расписание маршрутов с названиями аэропортов и самолетов. Поля: route_code, flight_no, departure_time, flight_hours, transit_stop_count, start_airport_code, start_airport_name, end_airport_code, end_airport_name, airplane_airport_code, airplane_code, airplane_name.

Представление `v_airports_departures_agg`: Содержит агрегированные сведения по аэропортам отправления. Поля: `airport_code`, `airport_name`, `departure_routes` — количество исходящих маршрутов, `total_transit_stops` — общее количество транзитных остановок по этим маршрутам.

2.4. Реализация клиентского приложения

Архитектурно приложение разделено на два основных модуля: `app.py` и `db.py`. Такое разделение позволяет выделить пользовательскую логику и работу с базой данных в отдельные уровни, что упрощает сопровождение и развитие программы.

Модуль `app.py` реализует пользовательский интерфейс и сценарии взаимодействия с системой. В нем размещены функции вывода данных в табличном виде, построения SQL-запросов на выборку, обработки команд пользователя, работы с отдельными таблицами, создания маршрутов с транзитными остановками, а также формирования отчетов. Главная функция `main()` обеспечивает запуск приложения, подключение к базе данных и отображение основного меню.

Модуль `db.py` реализует класс `Database`, который инкапсулирует доступ к PostgreSQL через библиотеку `psycopg2`. Данный класс предоставляет методы подключения к базе данных, выполнения SQL-команд, получения выборок, чтения метаданных таблиц, определения первичных ключей, а также универсальные операции вставки, обновления и удаления строк. Таким образом, вся низкоуровневая логика работы с СУБД вынесена в отдельный слой.

Подключение к базе данных выполняется при запуске программы. Строка подключения передается через переменные окружения `DATABASE_URL` или `PG_DSN`. Если параметры подключения отсутствуют, приложение выводит сообщение с примером корректного формата строки подключения и завершает работу. Такой способ конфигурации позволяет не хранить параметры доступа непосредственно в исходном коде.

Для отображения результатов запросов используется функция `_print_rows()`. Она принимает список строк в виде словарей и выводит их в консоль в виде форматированной таблицы. Ширина столбцов вычисляется автоматически на основе длины заголовков и содержимого. Если значение превышает допустимую ширину, строка сокращается с добавлением символа многоточия. Это делает вывод данных компактным и удобным для чтения.

Формирование SQL-запросов для просмотра таблиц реализовано в функции `_build_select()`. Она строит выражение `SELECT` с учетом выбранных столбцов, условий фильтрации, сортировки, ограничения количества строк и смещения. Такой подход позволяет

организовать постраничный просмотр данных и динамически изменять параметры выборки без ручного написания SQL-запросов для каждой таблицы.

Работа с таблицами выполняется в режиме `_table_mode()`. При входе в этот режим приложение получает список столбцов таблицы и сведения о первичных ключах. После этого загружается текущая выборка строк, которая отображается пользователю. В интерфейсе предусмотрены команды навигации по строкам и страницам, обновления данных, поиска, фильтрации, сортировки, добавления, редактирования и удаления записей. Это фактически универсальный CRUD-интерфейс для таблиц базы данных.

Добавление новой записи выполняется по команде `a`. Пользователю последовательно предлагается ввести значения для всех столбцов таблицы. Если поле оставлено пустым, используется значение `NULL` или значение по умолчанию. Для автоинкрементных первичных ключей ввод не требуется. После ввода данных вызывается метод `insert_row()` класса `Database`, который формирует SQL-запрос `INSERT ... RETURNING *` и возвращает добавленную запись для отображения пользователю.

Редактирование записи выполняется по команде `e`. Для изменения используется текущая выбранная строка. Если таблица не содержит первичного ключа, редактирование запрещается, так как невозможно однозначно определить нужную запись. При редактировании пользователю показываются старые значения полей, а новые значения вводятся только для тех атрибутов, которые нужно изменить. Обновление выполняется методом `update_row()`, который формирует SQL-запрос `UPDATE` с условием по первичному ключу.

Удаление записи выполняется по команде `d`. Для удаления также используется значение первичного ключа текущей строки. После подтверждения операции вызывается метод `delete_row()`, который выполняет SQL-запрос `DELETE FROM ... WHERE ....` Это обеспечивает безопасное удаление только выбранной пользователем записи.

Отдельный сценарий реализован в функции `_create_route_with_transit()`. Он предназначен для ввода составного объекта предметной области: маршрута и связанных с ним транзитных стоянок. Сначала пользователь вводит основные параметры маршрута, включая коды начального и конечного аэропортов, длительность полета, код самолета, дату и время вылета, номер рейса и примечания. Затем в рамках одной транзакции создается запись в таблице `routes`. После успешного создания маршрута пользователь может последовательно добавлять связанные записи в таблицу `transit_routes`, указывая аэропорт стоянки, номер стоянки, время прибытия и время отправления. Использование транзакции гарантирует

целостность данных: либо сохраняются и маршрут, и все введенные стоянки, либо в случае ошибки изменения не фиксируются.

Для аналитической работы в приложении реализован модуль отчетов `_reports_menu()`. Он предоставляет доступ к нескольким специализированным запросам к базе данных.

Отчет `_report_routes_to_destination()` предназначен для получения списка возможных маршрутов до выбранного конечного пункта назначения, включая маршруты с транзитными остановками. Пользователь указывает код конечного аэропорта и параметры сортировки. После этого выполняется SQL-запрос, возвращающий подходящие маршруты в требуемом порядке.

Отчет `_report_upcoming_flights()` формирует список ближайших рейсов. Пользователь может задать фильтр по аэропорту отправления, временное окно в часах и параметры сортировки. В запросе используются текущие дата и время, а также вычисляемое поле предполагаемого времени прибытия `eta`, определяемое как сумма времени отправления и продолжительности полета.

Отчет `_report_personnel_by_service()` выводит сведения о персонале по службам и должностям. Он позволяет получить сгруппированные данные о количестве сотрудников, числе задействованных бригад и среднем количестве работников на одну бригаду. Для формирования результата используется общее табличное выражение `WITH base AS (...)`, в котором предварительно вычисляются агрегированные показатели. Затем они дополняются наименованиями служб и должностей через соединение со справочными таблицами.

Класс `Database`, реализованный в модуле `db.py`, играет ключевую роль в организации взаимодействия приложения с базой данных. При создании объекта данного класса выполняется подключение к PostgreSQL с использованием `psycopg.connect()`. Параметр `row_factory=dict_row` обеспечивает возврат строк результата в виде словарей, что упрощает последующую обработку данных в Python-коде.

Метод `query()` предназначен для выполнения запросов `SELECT` и возврата списка строк. Метод `execute()` используется для выполнения команд, не требующих немедленного получения всех результатов. Метод `execute_script()` позволяет выполнять SQL-скрипты, предварительно разделяя их на отдельные инструкции. Для этого применяется вспомогательная функция `_split_sql_statements()`, которая корректно разбивает текст по символу `;`, учитывая нахождение внутри строковых литералов и идентификаторов в кавычках.

Для получения структуры таблиц реализован метод `table_columns()`. Он обращается к представлениям `information_schema.columns`, `information_schema.key_column_usage` и `information_schema.table_constraints`, благодаря чему приложение может автоматически

получать список полей, типы данных, признаки обязательности, значения по умолчанию и информацию о принадлежности к первичному ключу. Это позволяет строить универсальный интерфейс, не зависящий от жестко прописанной структуры каждой таблицы.

Методы `primary_keys()` и `primary_key()` предназначены для определения первичных ключей таблиц. Первый метод возвращает список столбцов, входящих в состав ключа, а второй используется в случае, если ключ состоит только из одного столбца. Эти методы необходимы для корректной реализации операций изменения и удаления данных.

Метод `insert_row()` принимает имя таблицы и словарь значений, после чего формирует параметризованный SQL-запрос на вставку данных. Использование параметров вместо прямой подстановки строк делает выполнение запроса безопаснее и предотвращает ошибки, связанные с форматированием значений. Аналогичным образом работает метод `update_row()`, который обновляет только те поля, которые действительно были изменены, и метод `delete_row()`, удаляющий строку по значению первичного ключа.

Таким образом, клиентское приложение представляет собой консольную информационную систему с универсальным интерфейсом работы с таблицами, поддержкой составных операций и встроенными отчетами. Приложение обеспечивает базовые функции администрирования данных и прикладной работы пользователя, а его реализация опирается на четкое разделение логики интерфейса и слоя доступа к данным. Это делает программу удобной для использования, расширения и сопровождения.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была спроектирована и реализована информационная система для хранения, обработки и управления данными в области транспортных перевозок. В процессе выполнения работы был проведён анализ предметной области, определены основные требования к разрабатываемой системе, выделены ключевые сущности и установлены связи между ними. На основе проведённого анализа была разработана структура базы данных в СУБД PostgreSQL, обеспечивающая надёжное хранение информации и поддержание её целостности.

В рамках практической реализации были созданы таблицы базы данных, отражающие основные объекты предметной области, разработаны SQL-скрипты для создания, заполнения и сопровождения базы данных, а также предусмотрены ограничения целостности и связи между таблицами. Это позволило обеспечить корректность вводимых данных, минимизировать вероятность ошибок и создать основу для эффективной работы системы.

Также в ходе курсовой работы было разработано клиентское приложение на языке Python, предназначенное для взаимодействия пользователя с базой данных. Реализованное приложение обеспечивает выполнение основных операций по обработке данных, включая добавление, просмотр, редактирование и удаление записей. Кроме того, были реализованы функции, связанные с организацией маршрутов, учётом остановочных пунктов и получением необходимой информации из базы данных. Архитектура приложения была выстроена с учётом разделения на модули, отвечающие за подключение к базе данных, обработку пользовательских действий, реализацию бизнес-логики и выполнение SQL-запросов.

Таким образом, результатом курсовой работы стала разработка полноценной информационной системы, включающей базу данных и клиентское программное приложение. Созданное решение позволяет автоматизировать основные процессы работы с данными в рассматриваемой предметной области, демонстрирует практическое применение полученных в ходе обучения знаний и может служить основой для дальнейшего совершенствования и расширения функциональных возможностей системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) PostgreSQL, Документация PostgreSQL 16 [Электронный ресурс] – URL: <https://www.postgresql.org/docs/16/>
- 2) Psycopg , Psycopg 2.9.12 documentation [Электронный ресурс] – URL: <https://www.psycopg.org/docs/>
- 3) Академия Selectel, Как работать с PostgreSQL в Python [Электронный ресурс] – URL: <https://selectel.ru/blog/tutorials/postgresql-python/>
- 4) Microsoft, Краткое руководство. Использование Visual Studio Code для подключения и запроса База данных SQL [Электронный ресурс] – URL: <https://learn.microsoft.com/ru-ru/azure/azure-sql/database/connect-query-vscode?view=azuresql>
- 5) Хабр, Введение в SQL & СУБД на примере доступа к данным через Python / Хабр [Электронный ресурс] – URL: <https://habr.com/ru/articles/783260/>
- 6) GitHub, Learn how to use PostgreSQL and Structured Query Language (SQL) to store and query your data. [Электронный ресурс] – URL: <https://github.com/dwyl/learn-postgresql>